Project 6 (C++): Given a file contains a list of sorted integers in ascending order. The task is to construct a balanced binary tree from the input sorted list. 1D vector will be used to store the input data.

What you have to do:
*** You will be given three data files: data1, data2, data3. data1 contains an out of order data; data2 has fewer integers; data3 has more integers.
1) Implement your program according to the specs below.
2) debug your program until it passes compilation.
3) Run and debug your program with data1 until your program outputs the error message.
4) Run and debug your program with data 2 until the result of inOrder (…) is identical to data2.
5) Run your program with data3 until the result of the inOrder (…) is identical to data3.

Include in your hard copy
- a cover page
- source code
- data1 with caption: "** Below is input data1"
- outFile for data1 with caption: "** Below is the outFile from data1."
- logFile for data1 with caption: "** Below is the logFile for data1." // Limit to 3 pages if more.

- data2 with caption: "** Below is input data2"
- outFile for data2 with caption: "** Below is the outFile from data2."
- logFile for data2 with caption: "** Below is the logFile for data2." // Limit to 3 pages if more.

- data3 with caption: "** Below is input data3"
- outFile for data3 with caption: "** Below is the outFile from data3."
- logFile for data3 with caption: "** Below is the logFile for data3." // Limit to 3 pages if more.

****

Language: C++

****

Project Name: Build a binary tree from an 1D array
Project points: 10 pts
Due Date: Soft copy (*.zip) and hard copies (*.docx):
10/10 (on time): 10/24/2026 Saturday before midnight (11:59pm).
-10/10 (non-submission): 10/24/2026 Saturday after midnight

All submission MUST include soft copy (*.zip) and hard copy (word file) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected. -10/10.

Email subject: (313) first name last name <Project 6: Build a binary tree from an 1D Vector (C++)>

*** Inside the email body includes:
- Your answer to the five questions given at the end of the project submission requirements. (-1 if omitted.)
- Screen recoding link. (-2 if omit screen recording).

*** Place your screen recording in your project submission email body below the 5 questions.

****

I. Input: inFile (use argv [1]): a text file contains a list of sorted positive integers in ascending order.

****

II. Outputs:
a) outFile (use argv [2]: as program dictates.
b) logFile (use argv [3]: as program dictates

****
III. Data Structures
****
- treeNode class
    (int) data
    (treeNode*) left
    (treeNode*) right

  Method:
    Constructor (…) // create a new treeNode.
- binTree class
  - (treeNode*) root
  - vector<int> dataAry;

  Methods:
  - loadData (…) // read data from inFile and push data into dataAry.
      // re-use code from your project 5; if you didn't do project 5, read the method in the project 5 specs.

  - (bool) checkData (dataAry) // If there is a data in dataAry is out of order,
      // outputs an error message to outFile: "ERROR! The input data is out of order at index =
      // integer =" print the index and the integer; then returns false; returns true otherwise.
      // The method prototype:
      // bool checkData(const vector<int>& dataAry)

  - printAry (…) // output the entire dataAry in the following format. Re-use code from project 5.

Index	data
0	8
1	9
2	10
3	15
4	23
:	:
:	:

  - (treeNode*) BuildBinTree (…) // Builds a binary tree from input data. **See algorithm below**.
  - preOrder (…) // **See algorithm below**. (print the entire tree, one data per text-line.
  - inOrder (…) // On your own.
  - postOrder (…) // On your own.

****
IV. main (…) #include <vector>
using namespace std;
****
Step 0: check if argc count is correct and all files can be opened.
  inFile ← open input file from argv [1]
  outFile ← open from argv [2]
  logFile ← open from argv [3]
step 1: loadData (inFile, dataAry, logFile)
step 2: (bool) flag ← checkData (dataAry)
  if flag == false
    exit (-1)

step 3: outFile ← "** The content of dataAry loaded from inFile **" //one text line
outFile ← "Index data" // one text line
outFile ← "==========" // one text line
printDataAry (dataAry, outFile)

step 4: int count ← dataAry.size()

Step 5: root ← BuildBinTree (dataAry, 0, count-1, logFile)

Step 6: outFile ← "*** Below is the preOrder of the binary tree ***"
preOrder (root, outFile)

Step 7: outFile ← "*** Below is the inOrder of the binary tree ***"
inOrder (root, outFile)

Step 7: ← "*** Below is the postOrder of the binary tree ***"
postOrder (root, outFile)

Step 8: close all files

****
V. (treeNode*) buildBinTree (inAry, leftIndex, rightIndex, logFile)
****

Step 0: logFile ← "*** Enter buildBinTree(…); leftIndex = rightIndex =" // fill in the value
Step 1: if leftIndex > rightIndex
return null

Step 2: midIndex ← (leftIndex + rightIndex + 1) / 2
size ← rightIndex – leftIndex + 1
newNode ← get a new treeNode with (-1, null, null)
logFile ← "midIndex = size =" // fill in value.

Step 3: if size == 1
newNode's data ← inAry[leftIndex]
else
newNode's data ← inAry[midIndex]
newNode's left ← buildBinTree (inAry, leftIndex, midIndex -1)
newNode's right ← buildBinTree (inAry, midIndex +1, rightIndex)

Step 4: logFile ← "*** Leaving buildBinTree(…); newNode's data is" // write value.

Step 5: return newNode

****
VI. preOrder (node, outFile)
****

if node != null
outFile ← node's data // per text-line
preOrder (node's left, outFile)
preOrder (node's right, outFile)